



Universidad Nacional de la Matanza
Departamento de Ingeniería e Investigaciones
Tecnológicas

Programación Concurrente
(3658)

Curso: 01-3900

Grupo M5

Integrantes:

- Ghiano, Gonzalo Agustín
- Minaudo, Lucas Agustín
- Polito, Thiago Martín
- Gomez, Matías Alejandro
- De Luca, Leonel Maximiliano

Link a repositorio de GitHub:

[2025-PROGC-Q2-M5/TP-Integrador/deathrun at develop · UNLAM-PROG-C/2025-PROGC-Q2-M5](#)

Descripción (Readme):

DeathRun Game

Juego multijugador desarrollado en Godot 4.5.1.

Autores

- * Ghiano, Gonzalo Agustín
- * Minaudo, Lucas Agustín
- * Polito, Thiago Martín
- * Gomez, Matías Alejandro
- * De Luca, Leonel Maximiliano

Requisitos del Sistema

- **Sistema Operativo:** Windows 10/11 (64 bits)
- **Arquitectura:** x86_64
- **Red:** Conexión LAN para modo multijugador en red local

Instalación

1. Descarga el archivo .exe del juego
2. Coloca el ejecutable en la carpeta de tu preferencia
3. No requiere instalación adicional

Cómo Jugar

El juego requiere dos instancias para funcionar: un **Host** (servidor) y un **Cliente**.

Opción 1: Dos jugadores en la misma computadora

Útil para pruebas o jugar localmente.

1. Ejecuta el .exe dos veces (se abrirán dos ventanas del juego)
2. En la primera ventana, crea el Host
3. En la segunda ventana, conéctate como Cliente usando la dirección:
127.0.0.1

Esta es la dirección de loopback, que apunta a tu propia máquina.

Opción 2: Dos jugadores en la misma red LAN

Para jugar en computadoras diferentes conectadas a la misma red.

En la PC que será el Host:

1. Ejecuta el .exe
2. Crea el Host
3. Obtén tu dirección IPv4 privada (ver instrucciones abajo)
4. Comparte esta dirección con el otro jugador

En la PC que será el Cliente:

1. Ejecuta el .exe
2. Conéctate como Cliente
3. Ingresa la dirección IPv4 privada del Host

Cómo obtener tu dirección IPv4 privada (Windows)

1. Abre el Símbolo del sistema (CMD):
 - Presiona Win + R
 - Escribe cmd y presiona Enter

2. Ejecuta el comando: ipconfig
3. Busca el adaptador de red activo (generalmente "Adaptador de Ethernet" o "Adaptador de LAN inalámbrica Wi-Fi")
4. Copia el valor de Dirección IPv4, que tendrá un formato similar a:
192.168.x.x o 10.x.x.x

Información Técnica

Motor: Godot 4.5.1

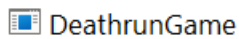
Plataforma: Windows Desktop

Arquitectura: x86_64

Protocolo de red: ENet (UDP)

Manual de Usuario

Para poder ejecutar la solución debe de descomprimir el archivo .zip que se encuentra en el repositorio, denominado DeathRunGame.zip, ahí encontrará el ejecutable .exe.



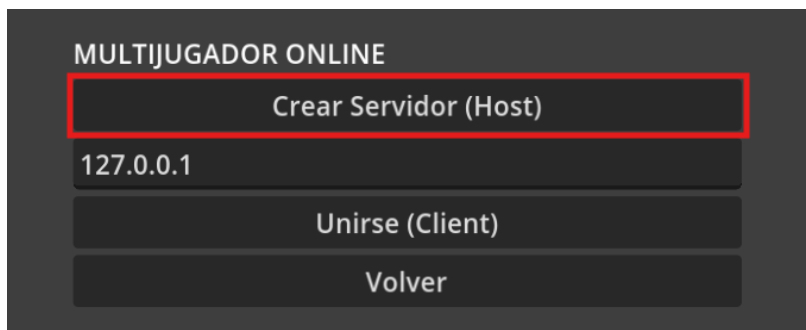
Este ejecutable va a abrir el juego, para jugar singleplayer, simplemente presione el botón "Un jugador" y arranque a jugar.



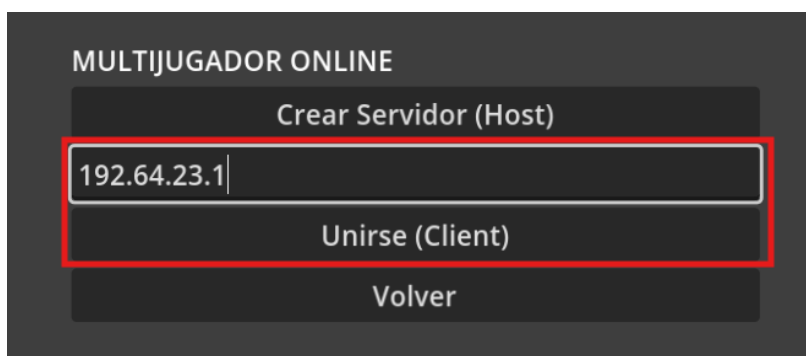
En cambio, para jugar de a dos jugadores deberá de ejecutar dos veces el .exe. y presionar “Multijugador”.



Luego en uno de los juegos deberá de presionar “Crear Servidor (Host)”, y darle la ip privada al otro jugador.



Y el otro jugador deberá de ingresar la ip del Host y presionar “Unirse (Client)” o en caso de ser en la misma PC usar la ip 127.0.0.1, que es la ip de loopback, para recibir en la misma PC., iniciando la partida.



Conclusiones

En la solución usamos un sistema de multijugador nativo de Godot, denominado ENetMultiplayerPeer, que nos permite realizar comunicación UDP con herramientas adicionales como entrega fiable y reordenamiento de paquetes. Esto nos permitió poder tener una comunicación segura pero fluida sin la latencia propia de los algoritmos de TCP. En particular, estamos creando el servidor en un jugador que lo denominamos Host (el runner), que deberá de sobrevivir a las trampas y llegar a la meta. En este host se realizó un enfoque autoritativo, donde toda la lógica de decisión de la aplicación se realiza en este host. Por otro lado, hay un único otro jugador, denominado Client (el trampero), el cual se conectará al host para comenzar la partida.

Estos dos jugadores, se van a comunicar mediante el uso de comunicación RPC, la cual permite ejecutar funciones entre dos instancias del juego en diferentes PCs, permitiendo comunicar dos procesos pesados desde una arquitectura distribuida.

Además, utilizamos hilos como mecanismos de concurrencia interna, los cuales fueron usados con el objetivo de alivianar la carga del hilo principal, realizando validaciones internas.

```
var rpc_worker_thread: Thread
# Cola de mensajes RPC
var rpc_queue: Array = []
var queue_mutex: Mutex
var queue_semaphore: Semaphore
```

Los mensajes por validar se ponen en una cola, que, al ser un recurso compartido entre los hilos, se controla su acceso vía un mutex. Además, para evitar la espera activa de los hilos worker, se utilizó un semáforo que permita despertarlo bajo demanda.